

Session 7 · Deck 7.A2

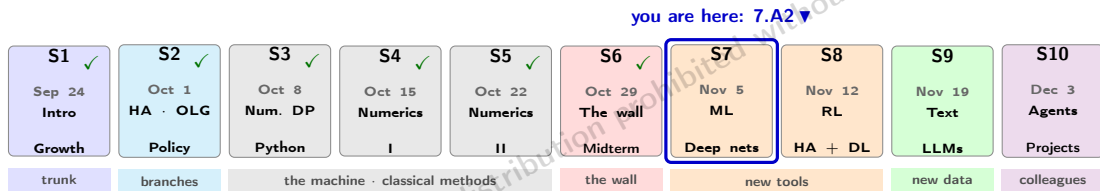
Neural Networks as Approximators

Zhigang Feng

Thursday, November 5, 2026

Build it on paper → solve it on the machine → push it to the frontier with AI

Where We Are



Build it on paper → solve it on the machine → push it to the frontier with AI

Block A: 7.A1 ML for macroeconomists · **7.A2** nets as approximators · 7.A3 training · 7.A4 architecture.

What We Are Building, and What We Will Ask of It

7.A1 ended on a sentence: a network is $\mathcal{N}(x; \theta)$, and the basis functions are built out of θ rather than chosen in advance.

This deck makes that concrete and then interrogates it, in that order:

1. **the object** — neuron, layer, composition, and why the nonlinearity between layers is load-bearing;
2. **the construction** — build an approximation out of kinks, by hand, and watch what changes when the kinks are allowed to move;
3. **the theorems** — what universal approximation promises, what Barron's rate promises, and what neither of them promises.

The comparison to keep

5.B2 asked “which basis?” and answered with orthogonality, conditioning, and a convergence rate you could bound. Everything in this deck is the same question asked of an object that will not give you any of those three — and the honest accounting of what you get instead.

A Neuron Is a Regression with a Nonlinear Output

One unit does two things. First a **net input**,

$$z = \mathbf{w}^\top \mathbf{x} + b,$$

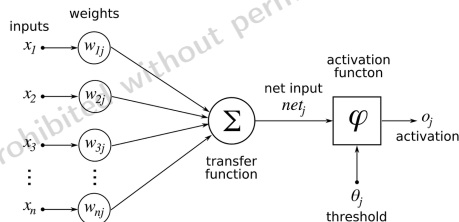
which you have written a thousand times — it is a linear index. Then an **activation**,

$$a = \sigma(z).$$

That is the whole unit. If σ is the logistic function, a single neuron is a logistic regression; if σ is the identity, it is OLS.

A neural network is not a new kind of object. It is many of these, arranged.

Feng & Miao, *AI for Economists*, ch. 9 §9.1.2.



The vocabulary is borrowed from biology and the mathematics is not. Nothing below depends on the metaphor, and it is safe to forget it.

Why the Nonlinearity Is Not Optional

An **affine** map is a linear map plus a constant — $\mathbf{W}\mathbf{x} + \mathbf{b}$, which is a layer with its activation stripped off. Suppose we drop σ and stack two of them:

$$\mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = \underbrace{(\mathbf{W}_2\mathbf{W}_1)}_{\text{one matrix}}\mathbf{x} + \underbrace{(\mathbf{W}_2\mathbf{b}_1 + \mathbf{b}_2)}_{\text{one vector}}.$$

The composition of two affine maps is an affine map. Stack a hundred and it is *still* an affine map — with a great many parameters and not one extra function it can represent.

Depth without nonlinearity buys exactly nothing. The activation is not a detail of implementation; it is the entire reason layers are worth stacking.

Say it as a warning

If a network is not learning and you have used a linear activation in the hidden layers, you have not built a deep model. You have built a very expensive linear regression.

The same identity read forwards is the useful part: each layer is an **affine map then a pointwise nonlinearity**,

$$f = \sigma_L \circ A_L \circ \cdots \circ \sigma_1 \circ A_1,$$

with $A_\ell(\mathbf{x}) = \mathbf{W}_\ell\mathbf{x} + \mathbf{b}_\ell$.

Layers, and the Object in One Line

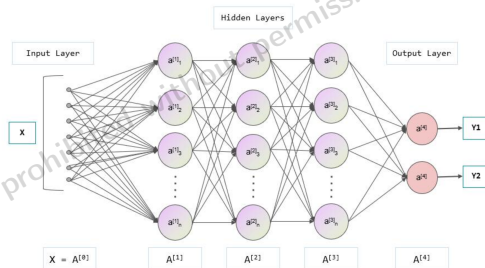
A **hidden layer** applies the same construction in parallel,

$$\mathbf{h} = \sigma(\mathbf{W}\mathbf{x} + \mathbf{b}),$$

with σ applied element by element. Compose L of them and read off the output:

$$\mathcal{N}(\mathbf{x}; \theta) = \mathbf{W}_{L+1} \sigma(\cdots \sigma(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) \cdots) + \mathbf{b}_{L+1}$$

θ collects every $\mathbf{W}_\ell$ and $\mathbf{b}_\ell$. **Within a layer, parallel; between layers, in series.**



The macro reading: successive layers are successive transformations of a state into a richer representation — which is what a dynamic program does to k_t on its way to a decision.

XOR: Not Approximated — Represented

The classical objection to a single layer of units is that it cannot represent the exclusive-or. It is true, and one hidden layer settles it **exactly**. Take

$$h_1 = \text{ReLU}(x_1 + x_2), \quad h_2 = \text{ReLU}(x_1 + x_2 - 1),$$

$$\hat{y} = h_1 - 2h_2.$$

x_1	x_2	h_1	h_2	$\hat{y}$	XOR
0	0	0	0	0	0
0	1	1	0	1	1
1	0	1	0	1	1
1	1	2	1	0	0

Read the last column

Not “close to”. **Equal**, in all four rows, at these exact weights. No linear map in (x_1, x_2) can do this, because the two classes are not linearly separable. **One hidden layer changes the coordinates until they are.**

This is the smallest complete answer to “what does depth buy?” — and it is checkable by hand in thirty seconds, which is why it is here.

Goodfellow, Bengio & Courville (2016) §6.1; Feng & Miao, *AI for Economists*, ch. 9 §9.2.3. Ledger xor: all four rows exact. Code: [s07_generate.py](#), [run_xor](#).

What Generality Costs, in Parameters

A value function in $d = 20$ states, and a network of three hidden layers of 256:

$$5,376 + 65,792 + 65,792 + 257 = \mathbf{137,217}.$$

Now count the alternatives at the same d :

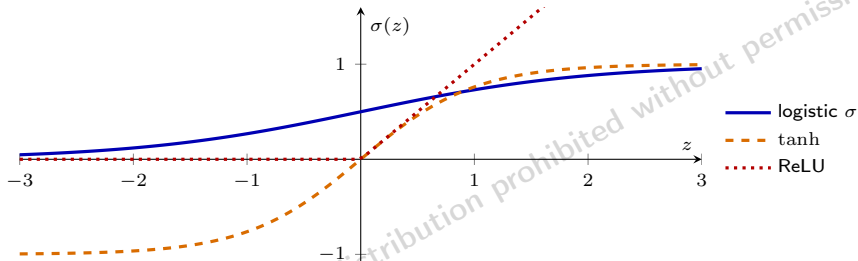
Object	count
the network above	137,217
complete polynomial, total degree ≤ 5	53,130
tensor-product polynomial, 5 per axis	3.7×10^{15}
tensor grid, 5 points per axis	9.5×10^{13}

Read rows two and three together

A **complete** basis keeps every monomial of *total* degree $\leq k$ — $\binom{d+k}{k}$ of them. A **tensor-product** basis keeps one degree- k factor per axis, $(k+1)^d$. Dropping to the complete basis turns 3.7×10^{15} into 53,130 — **5.B3's trick**, and the only reason a polynomial is in this table.

The network still carries **two and a half times** the complete basis's coefficients, so be suspicious rather than impressed. **Parameter count is the wrong axis**; accuracy *per* parameter is, and that is Barron's.

The Activations You Will Actually Meet

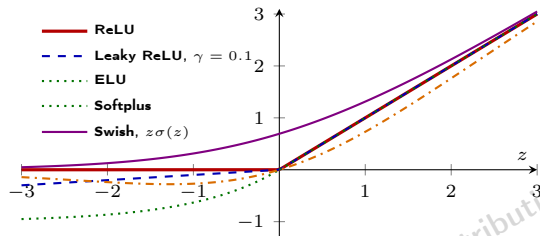


The logistic and tanh **saturate** — flat in both tails — and are relatives: $\tanh(z) = 2\sigma(2z) - 1$. Ranges $(0, 1)$ and $(-1, 1)$.

ReLU is **piecewise linear**: zero on one side, the identity on the other. It never saturates above, and its derivative is exactly 0 or 1.

Ledger activations. Code: `s07_generate.py`, `run_activations`.

The ReLU Family



Each fixes something about plain ReLU.

- **Leaky ReLU** gives the dead side a slope γ , so a unit that has switched off can come back.
- **ELU** does the same smoothly and saturates at -1 .
- **Softplus** is ReLU with the corner rounded off — and is **differentiable everywhere**, which matters when the model needs a second derivative.
- **Swish** and **GELU** ($z\Phi(z)$) are the modern defaults in large models; both dip slightly below zero.

Saturation, and Where the Vanishing Gradient Begins

The logistic derivative is $\sigma'(z) = \sigma(z)(1 - \sigma(z))$, whose **largest possible value is** $\sigma'(0) = 0.25$.

7.A3 shows the error signal is multiplied by σ' once per layer on the way back — so with logistic hidden units, by **at most** 0.25 each layer:

layers L	1	3	5	7
0.25^L , at best	0.25	0.016	0.00098	6×10^{-5}

And that is the *best* case; away from $z = 0$ it is far worse.

The consequence

By five layers the gradient reaching the first layer is a thousandth of the gradient at the last: **the early layers barely move**, and the network trains as though it were shallow.

ReLU's derivative is 1 on the active side — **no shrinkage at all**, so the signal survives depth. **That, not any approximation theorem, is why ReLU is the default for hidden layers.** **Two qualifications:** a unit off for every sample never returns (hence Leaky ReLU), and the sigmoid keeps the *output*, where a bound is wanted — 7.B1 uses it.

A Working Guide

Activation	Range	Main advantage	Typical use
ReLU	$[0, \infty)$	no saturation above; cheap; sparse activation	the default hidden unit
Leaky ReLU	$(-\infty, \infty)$	no dead units	when units are dying
ELU	$[-1, \infty)$	smooth, negative saturation	deeper stacks
logistic	$(0, 1)$	reads as a probability	a binary output; a bounded policy
tanh	$(-1, 1)$	zero-centred, smooth	shallow nets; recurrent layers
GELU, Swish	$\approx [-0.17, \infty)$	smooth, often trains better	large models
identity	$(-\infty, \infty)$	none — and that is the point	the output layer of a regression

The row that matters for Block B is the logistic one, used not as a probability but as a **bound**: $c = \sigma(\cdot) \times$ cash-on-hand is feasible by construction.

And the last row: putting an activation on the output of a regression silently bounds the prediction. It is a common and expensive mistake.

An Economist's Reading of the Activation

Choosing σ is choosing a **flexible functional form**. You have done it before — translog, CES, a semi-parametric index — and the considerations are recognisably the same: how much curvature, where, and at what cost in parameters.

There is one difference, and it is the whole difference. A translog is applied **once**. A network **composes** its functional form across layers, so what the second layer bends is not the data but the first layer's representation of it.

And the classical objection

“But θ has no interpretation.” Correct, and it is not meant to. 7.A1 separated the three jobs: this is **computation**, and the object being interpreted is the *function*, not the parameters. The policy function is what you report. Nobody reports a Chebyshev coefficient either.

The Construction: an Approximation Built out of Kinks

Take the simplest possible network: one hidden ReLU layer, one input, one output. Written out, it is

$$\hat{y}(x) = a_0 + \sum_{i=1}^N a_i \text{ReLU}(x - b_i).$$

Read the two sets of parameters separately, because they do quite different jobs:

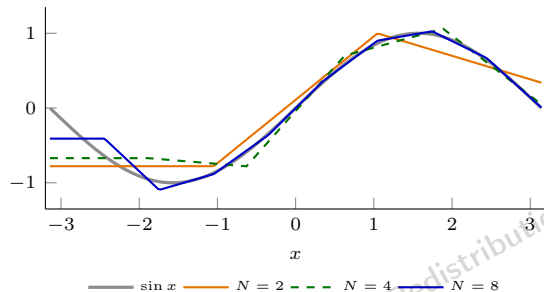
- the **shifts** b_i are the **kink points** — the places where the marginal slope is allowed to change;
- the **coefficients** a_i **scale** each piece's contribution, and so set the slope on each interval.

You have seen this basis

Fix the b_i in advance and $\hat{y}$ is **linear in the remaining parameters**. It is a regression on a piecewise-linear basis — 5.B4's elements, with the knots at the b_i .

Which means the next frame is not a new method. It is 5.B5's **least squares**, run on a basis you already know.

Worked: $\sin x$ out of Kinks, by Least Squares



The recipe is entirely familiar:

1. spread the b_i evenly over $[-\pi, \pi]$;
2. build a design matrix with columns $\text{ReLU}(x - b_i)$, plus a constant;
3. get the a_i by least squares.

N	unknowns	max err
2	3	0.7779
4	5	0.6704
8	9	0.4094
16	17	0.2220

There is no network here yet. With the shifts fixed this is a linear regression on a piecewise-linear basis — 5.B5's least squares on 5.B4's elements — and doubling N roughly halves the error, the $O(h)$ rate of a linear element.

The Turn: Let the Kinks Choose Where to Go

Now stop fixing the b_i . Train them **along with** the a_i — which is exactly what a one-hidden-layer ReLU network is, and nothing more.

N	fixed	learned	unknowns
2	0.7779	0.7263	$3 \rightarrow 5$
4	0.6704	0.7235	$5 \rightarrow 9$
8	0.4094	0.0389	$9 \rightarrow 17$
16	0.2220	0.0370	$17 \rightarrow 33$

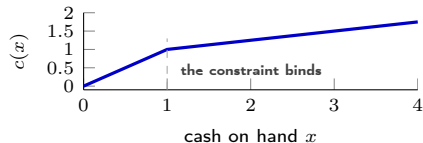
At $N = 8$ the error falls from 0.4094 to 0.0389 — a **factor of ten**, for twice the parameters.

This is the whole session

The basis is no longer chosen in advance. The approximant decides where the function needs to bend, and puts its kinks there. 5.B2 promised exactly this frame: “let the approximant learn where to bend — that is what a neural network does”.

Read row two honestly. At $N = 4$ the learned version is *worse* (0.7235 against 0.6704). Least squares is solved; training is a non-convex search returning a stationary point. **The flexibility is not guaranteed.**

And on a Function with a Kink in It



Same construction, new target:

$c(x) = \min\{x, 1 + 0.25(x - 1)\}$ — below the limit the household consumes everything, above it a fraction.

2.A1's kink, at $x = 1$.

Why the fixed version fails. Spread evenly, the two knots land at $x = 1.33$ and 2.67 , so a straight segment has to bend across the corner. And no N rescues it: evenly spaced knots almost never land on a kink.

N	max $ \text{err} $, knots		learned knot nearest $x = 1$
	fixed	learned	
2	0.7339	0.0027	1.0000
4	0.5090	0.0050	1.0000
8	0.2678	0.0065	1.0017

Look at the last column. With two kinks to spend, training puts one at 1.0000 — **the true kink, to four decimals**. Nobody told it where the constraint binds, and the error falls 274 $\times$.

Universal Approximation: What You Just Built

Universal approximation

Any continuous function on $[0, 1]^d$ can be approximated **uniformly**, to any accuracy, by a network with a *single* hidden layer:

$$\hat{f}(\mathbf{x}) = \sum_{j=1}^m v_j \sigma(\mathbf{w}_j^\top \mathbf{x} + b_j) + c.$$

Cybenko (1989) for sigmoidal σ ; Hornik, Stinchcombe & White (1989) more generally.

The nonlinearity is the glue: take σ away and the sum collapses to one straight line.

Do not be too impressed. Polynomials are dense in the continuous functions on a compact set too — Stone–Weierstrass, 5.B2's opening. Universality is not what distinguishes a network. **The rate is.**

The previous three frames *were* the construction: a weighted sum of shifted kinks bends to fit any curve. **One hidden unit is one building block; adding units shrinks the error.**

Four Things the Theorem Does Not Say

1. **Not how many units.** “There exists m ” — and m may be astronomically large. The theorem is silent on efficiency.
2. **Not how to find it.** It asserts that good weights *exist*. Finding them is the non-convex search of 7.A3, which promises only a stationary point.
3. **Not how much data.** Approximating a known function is not estimating an unknown one from a finite sample.
4. **Not that it will generalise.** Fitting the sample and fitting the function are different achievements.

Keep these apart

Approximation $\neq$ optimization $\neq$ generalization. Universal approximation speaks to the first only. 7.A3 is the second. The validation discipline of 7.B3 is the third.

The $N = 4$ row two frames ago is (2) in miniature: the better approximation existed — the fixed-knot fit found it — and training did not.

Feng & Miao, *AI for Economists*, ch. 9 §9.1.4. Which σ work? Leshno, Lin, Pinkus & Schocken (1993): a network is universal if and only if σ is not a polynomial — so the requirement is remarkably weak, and a polynomial activation is exactly the case that fails.

Barron's Theorem: a Rate That Does Not See d

In words. Double the hidden units and the squared error halves — and **that rate does not deteriorate as the number of state variables grows**, provided f is in the Barron class.

Barron (1993), Proposition 1

With m hidden units, on a ball B_r and averaging against μ ,

$$\int_{B_r} (f - \hat{f}_m)^2 d\mu \leq \frac{(2rC_f)^2}{m}, \quad C_f = \int \|\omega\| |\hat{f}(\omega)| d\omega.$$

C_f is f 's total **wiggleness**, weighted by frequency; the **Barron class** is the functions whose Fourier mass sits at low frequencies.

The bound is $O(1/m)$ with d **absent from it**. A fixed series basis manages only $O(m^{-2/d})$.

Barron (1993), *IEEE Trans. Inf. Theory*, Prop. 1; Feng & Miao, *AI for Economists*, ch. 9, Thm. 9.2.

Why the difference

A grid basis is fixed: resolution k per axis needs k^d pieces, most of them parked where f barely moves. That is the curse.

A neuron is a ridge $\sigma(\mathbf{w}^\top \mathbf{x} + b)$ — a soft step along a *learned* direction $\mathbf{w}$. The network spends its m units where f actually varies.

That is the mechanism you watched find the kink at 1.0000, now stated as a theorem.

Where the Dimension Actually Went

It is tempting to read $O(1/m)$ as “neural networks defeat the curse of dimensionality”. **That reading does not survive contact with the constant.**

$C_f = \int \|\omega\| |\hat{f}(\omega)| d\omega$ is itself an integral over $\mathbb{R}^d$. For many ordinary functions it grows with d ; for many others it is infinite, which puts them outside the class.

The dimension did not disappear. It moved house — into the constant, and into the narrowness of the class.

What is genuinely true

Where C_f stays moderate as d grows, a network's accuracy per parameter degrades far more gracefully than a tensor basis. **That is real and useful** — and it is not the statement “the curse is beaten”. So: a network is right when d is large *and* a few ridge directions express the function. **Whether that holds for your model is empirical**, which is why 7.B3 measures.

Feng & Miao, *AI for Economists*, ch. 9 §9.1.5. Barron (1994), *Machine Learning*, Thm. 3 adds the estimation side: risk $O(C_f^2/m) + O((md/n) \log n)$, optimised at $m \asymp C_f \sqrt{n/(d \log n)}$.

The Sharper Result: What a Fixed Basis Cannot Do

Barron's **Theorem 6** is the one that does the real work, and it is a *lower* bound. On the same Barron class, **any** approximation built from a basis chosen *in advance* — polynomials, splines, wavelets, Chebyshev, any of them — cannot do better than

$$O(m^{-2/d}).$$

Read that against the network's $O(1/m)$. The gap is not between polynomials and networks as such. **It is between a basis fixed before you look at f and a basis adapted to f .**

Barron (1993), Thm. 6, p. 942; Feng & Miao, *AI for Economists*, ch. 9 §9.1.5.

The through-line of this course

5.B2 chose a basis. 5.B3 chose a better one. 5.B4 chose where to break it. Every one of those choices was made **before** the solution was known — and Theorem 6 says that is precisely what caps the rate.

So the theorem names the thing to give up. It does not promise that giving it up will go well, and the next frames measure a case where it does not.

The Rates, Side by Side

Function class	Approximator	Size needed for accuracy ε	Source
k -times smooth	tensor-grid polynomial	$\varepsilon^{-d/k}$	Mhaskar (1996)
k -times smooth	any m -parameter continuous method	no better than $m^{-k/d}$	DeVore, Howard & Micchelli (1989)
Barron class	any fixed basis	no better than $m^{-2/d}$	Barron (1993), Thm. 6
Barron class	one hidden layer	$m \geq (2rC_f/\varepsilon)^2$ — no d in the exponent	Barron (1993), Prop. 1
compositional (binary tree)	a matching deep network	$O((d-1)\varepsilon^{-2/k})$ vs. shallow $O(\varepsilon^{-d/k})$	Poggio et al. (2017)

Rows one and two are Session 5's world, and row two is the discouraging one: the $m^{-k/d}$ ceiling binds on *any* method with a fixed parameterisation, not just on the ones we tried.

Rows three to five buy their escape with an **assumption** — membership of the Barron class, or a compositional structure. Neither is free, and neither is checkable for a value function you have not solved for.

Depth: the Other Axis

Everything so far has bought accuracy with **width** — more units in one layer. Depth is a different purchase, and for some functions a far better one.

Consider a function built as a **binary tree of two-variable compositions** — f is a function of pairs, of pairs, of pairs. A shallow network needs $O(\varepsilon^{-d/k})$ units; a deep network **whose architecture matches the tree** needs $O((d-1)\varepsilon^{-2/k})$.

The exponent stops depending on d . **But only if the function really has that structure** and the network is shaped like it.

Theorem (Eldan & Shamir 2016)

There is a function on $\mathbb{R}^d$ that a **three-layer** network represents with $O(d^{19/4})$ units, and that **no two-layer** network can approximate with fewer than e^{cd} units. Polynomial against exponential. **Depth is not a matter of taste.**

Compare 7.A4's message: *architecture is an assumption about the target*. This is the same statement, proved.

Mhaskar (1996); DeVore, Howard & Micchelli (1989); Poggio, Mhaskar, Rosasco, Miranda & Liao (2017); Eldan & Shamir (2016), Thm. 1. Feng & Miao, *AI for Economists*, ch. 9 §9.1.6.

Why Composition Is Economically Natural

The compositional assumption is not an exotic one to make about an economic model. Two familiar cases:

A decision. Raw data $\rightarrow$ a small number of cognitive variables, beliefs and expectations $\rightarrow$ a choice. Three layers, and the middle one is genuinely low-dimensional — which is what the whole recursive-methods apparatus assumes when it writes down a state.

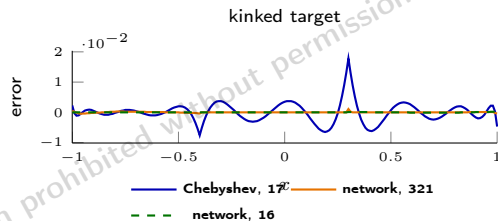
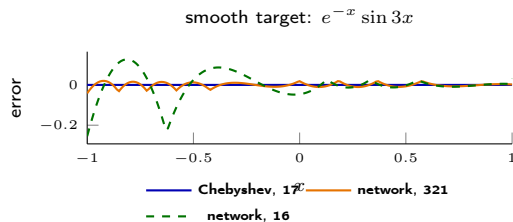
Aggregation. Individual decisions $\rightarrow$ group or sectoral outcomes $\rightarrow$ aggregates. Micro to macro *is* a composition, and the intermediate objects are the ones we name.

The connection worth making

Krusell–Smith asserts that the aggregate law of motion depends on Γ only through its **mean**. That is a compositional hypothesis — a one-dimensional bottleneck asserted by the modeller. S8's DeepHAM keeps the structure and **learns the bottleneck** instead of asserting it.

7.A1's manifold hypothesis, met a third time: the same idea in pixels, in states, and now in a theorem about depth.

Measured: a Network Against Chebyshev



Target	Method	Params	$\max \text{err} $
smooth	Chebyshev, degree 16	17	2.5×10^{-11}
smooth	network, 16–16	321	4.2×10^{-2}
smooth	network, 5 — matched	16	2.5×10^{-1}
kinked	Chebyshev, degree 16	17	1.8×10^{-2}
kinked	network, 16–16	321	1.1×10^{-3}
kinked	network, 5 — matched	16	2.0×10^{-4}

Ledger nn_vs_chheb; deviation from the target, not the level. Code: [s07_generate.py](#), [run_nn_vs_chheb](#).

Read That Table Before You Believe Anything Else

On the smooth function the network loses, badly.

Chebyshev with 17 coefficients reaches 2.5×10^{-11} — machine precision. The network with 321 parameters manages 4.2×10^{-2} : nineteen times the parameters and **nine orders of magnitude** worse. **Matched at 16 parameters it is worse again, 2.5×10^{-1} .**

On the kinked function it wins outright. Look at the right-hand panel: the Chebyshev error rings across the whole domain — 5.B2's Gibbs phenomenon — while the network's is flat.

The rule this gives you

Smooth and low-dimensional $\Rightarrow$ a spectral basis: faster, smaller, better. **Kinked, or high-dimensional** $\Rightarrow$ the adaptive basis earns its keep.

“But the network had more parameters”

So the last row of each block matches it: **16 against 17**. Matched, the verdict sharpens both ways — ten times worse on the smooth target, **88 $\times$ better on the kinked one**, beating even the 321-parameter net.

Kinks are not exotic in macroeconomics: a borrowing limit (2.A1), irreversible investment, a tax bracket, a zero lower bound.

Takeaways, and What Is Not Yet Settled

- A neuron is a regression with a nonlinear output; **without the nonlinearity, depth buys nothing at all.**
- Fix the kinks and a one-layer ReLU net *is* least squares on a piecewise-linear basis. **Let them move and it finds the constraint**, at 1.0000.
- Universal approximation is **existence**, not a recipe, and polynomials have it too.
- Barron's rate is free of d *in the exponent*; the dimension lives in C_f . **Theorem 6 is the sharp one:** a basis fixed in advance cannot beat $m^{-2/d}$.
- Measured: the network loses on a smooth target and wins on a kinked one.

Bridge to 7.A3

Every promise here was of the form “a good network *exists*”. Nothing has said how to find one. **7.A3 is the finding:**

backpropagation, a first-order optimizer, and the discipline that keeps the fit honest. Including the $N = 4$ row, which is what happens when the search does not deliver.